

Plan de implementación de la API del chatbox

Fastify · TypeScript · PostgreSQL/pgvector · Groq

Versión revisada - MVP sin Redis obligatorio

Este documento divide la construcción de la API en pasos cerrables. Cada paso tiene un objetivo, un alcance concreto y requisitos verificables. La API se implementará con Fastify, Node.js y TypeScript, PostgreSQL con pgvector, un modelo local de embeddings y Groq como proveedor de generación.

Principios del proyecto

- Construir primero un MVP útil y desplegable.
- Evitar infraestructura distribuida mientras exista una sola instancia.
- Mantener contratos sustituibles para poder añadir Redis más adelante.
- Separar ingesta, recuperación y generación para poder probar cada parte.
- No consumir Groq cuando no exista contexto suficiente.
- No almacenar conversaciones ni preguntas completas por defecto.

Decisiones iniciales

Área	Decisión
Runtime	Node.js LTS
Lenguaje	TypeScript en modo estricto
Framework HTTP	Fastify
Validación	JSON Schema integrado con Fastify
Persistencia	PostgreSQL + pgvector
Generación	Groq, únicamente desde el backend
Embeddings	intfloat/multilingual-e5-small, inicialmente
RAG	Pipeline directo, sin LangChain
Conversación	No persistente en el MVP
Identidad	UUID anónimo firmado en cookie
Streaming	Server-Sent Events sobre HTTP
Rate limiting inicial	Memoria, una sola instancia
Escalado posterior	Redis mediante adaptador sustituible

Alcance del MVP

Incluye:

- GET /health.
- Ingesta controlada mediante CLI.
- Embeddings locales y persistencia vectorial.
- Recuperación semántica.
- POST /api/chat con RAG.
- Streaming mediante SSE.
- Límites por sesión, IP y concurrencia.
- Presupuesto global básico.
- Logs técnicos y pruebas reproducibles.

Fuera de alcance inicialmente:

- Panel de administración y login.
- Memoria permanente de conversaciones.
- Agentes o herramientas externas.
- Indexación automática de GitHub.
- LangChain.
- Redis obligatorio.
- Escalado horizontal con varias instancias.

Paso 1 - Inicializar Fastify y el contrato técnico

Objetivo. Crear una API Fastify ejecutable localmente, con estructura, configuración, validación y contrato HTTP definidos.

Requisitos funcionales

- ☐ El proyecto se instala desde cero con un único comando documentado.
- ☐ Existe un comando de desarrollo, compilación y ejecución de producción.
- ☐ TypeScript usa modo estricto.
- ☐ La configuración se valida al arrancar.
- ☐ La API falla claramente si falta una variable obligatoria.
- ☐ Los errores públicos tienen un formato JSON estable.
- ☐ Fastify oculta stack traces internos en producción.
- ☐ El contrato documenta GET /health y POST /api/chat.
- ☐ La validación de entrada utiliza JSON Schema.

Cierre del paso. El proyecto instala, arranca y compila desde un entorno limpio. Una petición inválida produce un error JSON estable.

Paso 2 - Configurar PostgreSQL y el esquema vectorial

Objetivo. Disponer de una base de datos versionada para documentos, versiones, fragmentos y vectores.

Requisitos funcionales

- ☐ La conexión se configura externamente.
- ☐ La extensión vector se habilita de forma reproducible.

- ☐ Existen migraciones versionadas.
- ☐ Hay tablas para documentos y fragmentos.
- ☐ Cada fragmento guarda texto, embedding y metadatos JSON.
- ☐ La dimensión coincide con el modelo elegido.
- ☐ Solo una versión de cada fuente permanece activa.
- ☐ La eliminación no deja huérfanos.
- ☐ Se distinguen errores de conexión y consulta.

Cierre del paso. Una base vacía puede migrarse e insertar, consultar y eliminar un documento de prueba con sus fragmentos.

Paso 3 - Implementar la ingesta controlada

Objetivo. Convertir el currículum y documentos curados en fragmentos listos para búsqueda.

Requisitos funcionales

- ☐ Existe un comando CLI ejecutable en local y CI.
- ☐ No existe un endpoint público de ingesta en el MVP.
- ☐ Acepta Markdown y texto plano.
- ☐ Cada documento se identifica por fuente y versión.
- ☐ El contenido se divide primero por secciones semánticas.
- ☐ Cada fragmento conserva el título de sección.
- ☐ Los metadatos admiten URL, proyecto, idioma y categoría.
- ☐ Los embeddings se generan localmente.
- ☐ Repetir fuente y versión no duplica fragmentos.
- ☐ Una nueva versión sustituye atómicamente a la anterior.
- ☐ Un documento inválido no queda parcialmente publicado.
- ☐ El comando informa de documentos, fragmentos y errores.

Cierre del paso. El currículum puede ingerirse dos veces sin duplicados y una versión nueva reemplaza correctamente a la anterior.

Paso 4 - Implementar la recuperación semántica

Objetivo. Recuperar fragmentos relevantes sin invocar todavía a Groq.

Requisitos funcionales

- ☐ La pregunta usa el mismo espacio vectorial de la ingesta.
- ☐ El número de resultados es configurable.
- ☐ Cada resultado incluye texto, similitud y metadatos.
- ☐ Se puede filtrar por fuente o categoría.
- ☐ El orden es determinista en empates.
- ☐ Existe un umbral mínimo configurable.
- ☐ Sin resultados suficientes se indica falta de contexto.
- ☐ No se devuelven versiones inactivas.

☐ Existen pruebas con preguntas conocidas.

Cierre del paso. El conjunto de evaluación recupera los fragmentos esperados y rechaza preguntas sin evidencia.

Paso 5 - Implementar el chat con Groq y SSE

Objetivo. Generar respuestas fundamentadas y entregarlas progresivamente.

Requisitos funcionales

- ☐ Existe POST /api/chat.
- ☐ La pregunta se valida y tiene longitud máxima.
- ☐ La recuperación ocurre antes de Groq.
- ☐ El modelo solo puede usar el contexto proporcionado.
- ☐ El prompt exige reconocer falta de información.
- ☐ El contenido recuperado se trata como datos, no instrucciones.
- ☐ Sin contexto suficiente no se consume Groq.
- ☐ La clave de Groq solo existe en backend.
- ☐ Modelo, temperatura y tokens son configurables.
- ☐ La respuesta usa SSE.
- ☐ Se emiten eventos token, sources, done y error.
- ☐ La desconexión cancela el trabajo cuando sea posible.
- ☐ Timeouts y 429 no provocan reintentos ilimitados.

Cierre del paso. Una pregunta con evidencia produce una respuesta basada en fuentes; una pregunta sin evidencia no llama a Groq.

Contrato SSE inicial

```
event: token  
data: {"text":"..."}
```

```
event: sources  
data: {"sources":[{"title":"...", "url":"..."}]}
```

```
event: done  
data: {}
```

Paso 6 - Añadir identidad anónima y límites básicos

Objetivo. Evitar abuso razonable sin introducir Redis antes de necesitarlo.

Requisitos funcionales

- ☐ El backend genera un UUID aleatorio.
- ☐ El UUID se firma con HMAC y expira.
- ☐ La cookie es HttpOnly, SameSite=Lax y Secure en producción.
- ☐ Una firma inválida genera una identidad nueva.
- ☐ Existe límite por sesión y por IP.
- ☐ Solo se confía en proxies configurados como fiables.
- ☐ Existe una petición simultánea máxima por sesión.

- ☐ Hay límite de longitud y tokens.
- ☐ Los 429 incluyen Retry-After cuando proceda.
- ☐ Existe un presupuesto global básico.
- ☐ Los contadores iniciales se almacenan en memoria.
- ☐ El almacenamiento está detrás de una interfaz sustituible.
- ☐ Se documenta el reinicio de contadores al reiniciar la instancia.

Cierre del paso. Las pruebas demuestran los límites por sesión, IP y concurrencia, y que el presupuesto global bloquea nuevas llamadas.

Valores iniciales sugeridos

Límite	Valor inicial
Preguntas por minuto y sesión	5
Preguntas por día y sesión	30
Peticiones simultáneas por sesión	1
Longitud máxima	500 caracteres
Tokens máximos de respuesta	400

Paso 7 - Observabilidad y operación

Objetivo. Diagnosticar errores y controlar consumo sin almacenar contenido innecesario.

Requisitos funcionales

- ☐ GET /health comprueba que el proceso está vivo.
- ☐ Existe una comprobación separada de PostgreSQL.
- ☐ Cada petición tiene identificador de correlación.
- ☐ Los logs incluyen duración, endpoint, estado y causa resumida.
- ☐ No se registran claves, cookies ni credenciales.
- ☐ Las preguntas completas no se almacenan por defecto.
- ☐ Tokens y consumo se agregan cuando sea posible.
- ☐ Se distinguen errores de proveedor, base de datos y aplicación.
- ☐ Existen timeouts para base de datos, embeddings y Groq.
- ☐ El cierre espera peticiones activas durante un periodo limitado.

Cierre del paso. Los logs y salud permiten distinguir API caída, PostgreSQL no disponible, límite alcanzado y error de Groq.

Paso 8 - Pruebas, evaluación y despliegue

Objetivo. Verificar el flujo completo antes de publicar el widget Preact.

Requisitos funcionales

- ☐ Hay pruebas unitarias para validación, chunking, prompt y límites.

- ☐ Hay pruebas de integración para PostgreSQL y pgvector.
- ☐ Existe una prueba completa pregunta -> embedding -> recuperación -> respuesta.
- ☐ Se verifica que sin evidencia no se llama a Groq.
- ☐ Se prueban 429, timeout, desconexión y fallo de proveedor.
- ☐ Existe un conjunto de 15 a 30 preguntas reales.
- ☐ Los casos pueden definir fuentes y términos esperados.
- ☐ La aplicación funciona con Docker Compose.
- ☐ Las migraciones y variables están documentadas.
- ☐ CORS se restringe al dominio del portfolio.
- ☐ El endpoint no se publica hasta activar límites básicos.

Cierre del paso. El flujo funciona en un entorno reproducible y cubre éxito, falta de contexto, abuso y fallos externos.

Fase posterior - Redis y escalado horizontal

Esta fase no forma parte del MVP. Se inicia únicamente si la API se despliega con varias instancias, si los reinicios de contadores son un problema real o si el tráfico requiere coordinación distribuida.

- ☐ Implementar el adaptador de contadores con Redis.
- ☐ Usar operaciones atómicas y expiraciones.
- ☐ Compartir límites y presupuesto global entre instancias.
- ☐ Comprobar el comportamiento ante caída temporal de Redis.
- ☐ Añadir un indicador de salud separado para Redis.
- ☐ Validar el rate limiting con varias instancias concurrentes.

Orden recomendado de ejecución

1. Inicializar Fastify y el contrato técnico.
2. Configurar PostgreSQL y migraciones.
3. Implementar la ingesta.
4. Implementar la recuperación.
5. Implementar chat y SSE.
6. Conectar el widget Preact en local.
7. Añadir identidad y límites básicos.
8. Añadir observabilidad.
9. Completar pruebas, evaluación y despliegue.
10. Añadir Redis solo cuando exista una necesidad real.

Cada paso debe cerrarse antes de iniciar el siguiente. El frontend puede conectarse localmente al finalizar el paso 5, pero el endpoint no debe publicarse sin completar los límites básicos del paso 6.